Handling Missing Data in ETL Assignment

Objective:
This DPP helps understand:
Why missing data occurs in ETL pipelines
How different handling techniques impact analytics
How to choose the right method instead of blindly deleting data.

SECTION A – THEORETICAL QUESTIONS

Q1. What are the most common reasons for missing data in ETL pipelines?

Answer:

Missing data is one of the most frequent data quality issues encountered in ETL pipelines. It occurs when expected values are absent in one or more fields of a dataset. Below are the most common reasons:

1. Incomplete Data Entry at the Source

Data is often entered manually by users who may skip optional or non-mandatory fields, leaving them empty in the source system.

Example: A customer fills an online form but leaves the phone number field blank, resulting in a NULL value in the database.

2. System or Network Failures During Extraction

If the source system crashes, the network drops, or the ETL job times out mid-extraction, only a partial dataset gets pulled — leaving records incomplete or missing entirely.

Example: An ETL job extracting 1 million records fails at record 600,000 due to a timeout, leaving 400,000 records unextracted.

3. Schema or Structural Mismatches

When the source system adds a new column or changes its structure, the ETL pipeline may not recognize the new field and leave it as NULL in the target system.

Example: A source database adds a new discount_code column but the ETL mapping does not include it, so it arrives as NULL in the warehouse.

4. Data Migration from Legacy Systems

Old or legacy systems often have inconsistent or incomplete historical records. When migrated into modern systems, many fields simply have no data available.

Example: A company migrating 10-year-old sales records finds that early records have no email addresses since emails were not collected at that time.

5. Integration of Multiple Heterogeneous Sources

When combining data from different systems, some sources may not have equivalent fields for every column in the target — resulting in missing values for those fields.

Example: Sales data from one system has a region column but the merged data from another system does not, leaving the region field empty for those records.

6. Filtering or Transformation Errors

Bugs or incorrect logic in the transformation stage can accidentally drop or nullify values that were present in the extracted data.

Example: A transformation script incorrectly filters out records with a specific date format, causing valid records to be dropped from the final dataset.

Q2. Why is blindly deleting rows with missing values considered a bad practice in ETL?

Answer:

In ETL pipelines, blindly deleting rows with missing values means removing any record that contains a NULL or empty field without any analysis or consideration. Although it may seem like a quick fix, it is widely considered a bad practice for the following reasons:

1. Significant Data Loss

Deleting rows without analysis can remove a large portion of the dataset, especially when missing values are spread across multiple columns. This reduces the volume of data available for analysis and reporting.

Example: A dataset of 100,000 customer records has missing values in 5 different columns. Blindly deleting all affected rows could eliminate 40,000+ records, severely impacting analysis accuracy.

2. Introduction of Bias

When rows are deleted without understanding why the data is missing, it can create a biased dataset that no longer represents the true population or business reality.

Example: If all deleted records happen to belong to customers from a specific region, the remaining dataset underrepresents that region — leading to incorrect business conclusions.

3. Loss of Partially Useful Records

A row with one missing value out of 20 columns still contains 19 useful fields. Deleting the entire row wastes all the valid information it holds.

Example: An order record has all details filled — product, quantity, price, date — but is missing only the postal code. Deleting it loses all other valuable order information.

4. Impact on Machine Learning Models

In data science and machine learning pipelines, blindly removing rows reduces the training dataset size, which can lead to underfitting and poor model performance.

Example: A fraud detection model trained on a reduced dataset due to row deletions may miss important patterns present in the deleted records.

5. Masking Underlying Data Quality Issues

Deleting rows hides the real problem instead of fixing it. The root cause of missing data — such as a broken source system or faulty data entry — remains unaddressed.

Example: If a sensor consistently fails to record temperature values, deleting those rows hides the sensor malfunction instead of alerting the team to fix it.

Q3. Explain the difference between: Listwise deletion Column deletion Also mention one scenario where each is appropriate.

Answer:

Both Listwise Deletion and Column Deletion are techniques used to handle missing data in ETL pipelines. While both involve removing data, they differ in what exactly gets removed a row or an entire column.

Listwise Deletion

Listwise Deletion means removing an entire **row** from the dataset when one or more values are missing in that record.

Example:

Order_ID	Customer	Amount	Region
101	Rahul	5000	North
102	Priya	NULL	South
103	Amit	3000	NULL

After Listwise Deletion, rows 102 and 103 are removed entirely because they contain missing values.

When it is Appropriate: When the number of missing rows is very small (under 5% of total data) and the missing records are not critical to the analysis — so their removal does not affect the overall results or introduce bias.

Scenario: A sales dataset of 50,000 records has only 200 rows with missing order amounts. Since these represent less than 1% of the data, deleting them has minimal impact on reporting accuracy.

Column Deletion

Column Deletion means removing an entire **column** from the dataset when that column has too many missing values to be useful or reliable.

Example:

Order_ID	Customer	Amount	Discount_Code
101	Rahul	5000	NULL
102	Priya	3200	NULL
103	Amit	4100	NULL

Since the Discount_Code column is missing for almost all records, the entire column is dropped.

When it is Appropriate: When a column has a very high percentage of missing values (typically above 60-70%) and does not contribute meaningfully to the analysis or model — making imputation impractical or unreliable.

Scenario: A customer dataset has a Secondary_Phone column where 80% of records have no value. Since the column is rarely filled and not required for analysis, it is dropped entirely from the pipeline.

Key Differences

Factor	Listwise Deletion	Column Deletion
What is Removed	Entire row	Entire column
When Used	Few rows have missing values	Column has too many missing values
Data Lost	Individual records	An entire feature / attribute
Risk	Losing valid records	Losing potentially useful information
Threshold	Missing rows under 5%	Missing values in column above 60-70%

Q4. Why is median imputation preferred over mean imputation for skewed data such as income?

Answer:

When handling missing values in numerical columns during the Transform stage of ETL, choosing the right imputation method is critical. For skewed data like income, salary, or house prices, median imputation is strongly preferred over mean imputation for specific and important reasons.

What is Mean Imputation?

Mean imputation replaces missing values with the **average** of all existing values in the column.

Example:

Income values: 20,000 | 25,000 | 22,000 | 23,000 | 500,000 (outlier)

Mean = (20,000 + 25,000 + 22,000 + 23,000 + 500,000) / 5 = **118,000**

The mean of 118,000 does not represent any typical income in this dataset — it is heavily pulled upward by the outlier of 500,000.

What is Median Imputation?

Median imputation replaces missing values with the **middle value** when all values are arranged in order.

Example:

Arranged: 20,000 | 22,000 | **23,000** | 25,000 | 500,000

Median = **23,000**

The median of 23,000 accurately represents the typical income in this dataset and is completely unaffected by the outlier.

Why Median is Preferred for Skewed Data

1. Not Affected by Outliers Median is a positional measure — it simply picks the middle value regardless of how extreme the other values are. Mean on the other hand is a mathematical average that gets pulled towards extreme values, making it unreliable for skewed distributions.

2. Better Represents Typical Value For income data, a few very high earners can massively inflate the mean. The median better represents what a typical person in the dataset actually earns.

Example: In a country where most people earn between 20,000 and 40,000 but a few billionaires earn crores, the mean income would be misleadingly high while the median would reflect the true middle-class reality.

3. Preserves Data Distribution Using median imputation keeps the overall distribution of the data intact without distorting it, ensuring downstream analysis and model training remain accurate.

4. Reduces Imputation Bias Replacing missing income values with an inflated mean introduces bias into the dataset — making the imputed values unrealistic for most records. Median avoids this by staying close to where most values actually cluster.

Q5. What is forward fill and in what type of dataset is it most useful?

Answer:

Forward Fill (also called **ffill**) is a missing value imputation technique where a missing value is replaced by the **last known previous value** in the same column. It carries the most recent valid value forward to fill the gap.

In simple terms — if a value is missing, look at the value just above it and copy it down.

Example:

Original Dataset:

Time	Temperature
9:00 AM	32°C
10:00 AM	NULL
11:00 AM	NULL
12:00 PM	35°C

After Forward Fill:

Time	Temperature
9:00 AM	32°C
10:00 AM	32°C
11:00 AM	32°C
12:00 PM	35°C

The missing values at 10:00 AM and 11:00 AM are filled with 32°C — the last known value before the gap.

How to Apply Forward Fill in Excel:

In Excel, forward fill can be done using a simple IF formula. In cell B3 write:

=IF(B3="", B2, B3)

This checks if the current cell is empty — if yes, it copies the value from the cell above it. Drag this formula down the column to apply it to all rows.

Alternatively in Excel you can use **Go To Special → Blanks → Fill Down** to forward fill all empty cells in a selected column in one step.

Where is Forward Fill Most Useful?

Forward Fill is most useful in **time-series datasets** where data is recorded sequentially over time and consecutive values are naturally related to each other.

1. IoT and Sensor Data — Sensors sometimes miss readings for a few intervals. The previous reading is the most logical estimate.

Example: A factory temperature sensor misses readings for 2 hours. Forward fill uses the last recorded temperature as the best available estimate.

2. Stock Market Data — On non-trading days like weekends or holidays, stock prices are not recorded. Forward fill carries the last closing price forward.

Example: A stock priced at ₹500 on Friday carries that value forward for Saturday and Sunday.

3. Weather Data — Meteorological stations occasionally miss hourly readings. Since weather changes gradually, the previous reading is a reasonable estimate.

4. Patient Health Monitoring — If a patient's vitals are not recorded for a few intervals, the last known reading is carried forward as the estimated current value.

Q6. Why should flagging missing values be done before imputation in an ETL workflow?

Answer:

Flagging missing values means creating an additional indicator column in the dataset that marks whether a value was originally missing or not — **before** any imputation is applied. Typically a new binary column is added where **1 = value was missing** and **0 = value was present.**

Example:

Customer_ID	Income	Income_Missing_Flag
101	50,000	0
102	NULL	1

Customer_ID	Income	Income_Missing_Flag
103	45,000	0
104	NULL	1

After flagging, imputation is applied to fill the NULL values while the flag column preserves the original missing information.

Why Flagging Must Be Done Before Imputation

1. Preserves the Original Missing Information

Once imputation is applied, the dataset no longer shows which values were originally missing. Flagging before imputation ensures this information is permanently recorded for future reference.

Example: After median imputation, all income values appear filled. Without a flag, there is no way to know which values were real and which were imputed.

2. Enables Better Analysis and Auditing

Flagged columns allow analysts and data auditors to separately analyze records that had missing values versus those that did not — helping identify patterns or problems in the source data.

Example: An analyst notices that flagged records consistently belong to customers from a specific region — indicating a data collection issue in that region that needs to be fixed at the source.

3. Improves Machine Learning Model Accuracy

In machine learning pipelines, the fact that a value was missing can itself be a meaningful signal. If flagging is not done before imputation, this signal is permanently lost and the model loses potentially useful information.

Example: In a loan default prediction model, customers with missing income data may have a higher default rate. The missing flag column captures this pattern even after imputation fills the values.

4. Supports Data Lineage and Transparency

ETL pipelines must maintain data lineage — a clear record of how data was processed and modified. Flagging documents exactly which values were imputed, ensuring full transparency in the pipeline.

5. Allows Easy Rollback and Reprocessing

If the imputation strategy needs to be changed later, flagged records can be easily identified and reprocessed. Without flags, it is impossible to distinguish original values from imputed ones.

How it is Done in Power BI:

In Power Query Editor, before applying Fill Down, add a custom column using:

= if [Income] = null then 1 else 0

This creates the flag column first, then Fill Down is applied to handle the missing values.

How it is Done in Excel:

Before filling missing values, add a new column with the formula:

=IF(B2="", 1, 0)

This marks all empty cells as 1 before any imputation is performed.

Q7. Consider a scenario where income is missing for many customers. How can this missingness itself provide business insights?

Answer:

Missing income data can itself provide valuable business insights because the absence of information often indicates a pattern in customer behaviour rather than being purely random.

For example:

1. **Customer Privacy Concerns**
   Customers who do not disclose their income may be more concerned about privacy and may be less willing to share personal information. This insight can help businesses design more trust-building strategies.
2. **Specific Customer Segments**
   Missing income values may be concentrated within certain age groups, regions, occupations, or customer categories. Identifying these segments can help businesses better understand their target audience.
3. **Application or Data Collection Issues**
   A high number of missing income records may indicate problems in the data collection process, such as unclear forms, optional fields, or technical issues during data entry.
4. **Potential Indicator of Financial Behavior**
   Customers who do not report income may exhibit different purchasing patterns,

spending habits, or product preferences compared to those who provide income information. Businesses can analyze these differences for targeted marketing.

5. **Risk Assessment Insights**
   In industries such as banking or lending, missing income information may be associated with higher uncertainty or risk, prompting organizations to apply additional verification procedures.

SECTION B – PRACTICAL QUESTIONS

Customer_ID	Name	City	Monthly_Sales	Income	Region
101	Rahul Mehta	Mumbai	12000	65000	West
102	Anjali Rao	Bengaluru	NaN	NaN	South
103	Suresh Iyer	Chennai	15000	72000	South
104	Neha Singh	Delhi	NaN	NaN	North
105	Amit Verma	Pune	18000	58000	NaN
106	Karan Shah	Ahmedabad	NaN	61000	West
107	Pooja Das	Kolkata	14000	NaN	East
108	Riya Kapoor	Jaipur	16000	69000	North

Q8. Listwise Deletion Remove all rows where Region is missing.

Tasks:

Answer:

1. Identify affected rows

Customer_ID	Name	City	Monthly_Sales	Income	Region
105	Amit Verma	Pune	18,000	58,000	NaN

2. Show the dataset after deletion

Customer_ID	Name	City	Monthly_Sales	Income	Region
101	Rahul Mehta	Mumbai	12,000	65,000	West
102	Anjali Rao	Bengaluru	NaN	NaN	South
103	Suresh Iyer	Chennai	15,000	72,000	South
104	Neha Singh	Delhi	NaN	NaN	North
106	Karan Shah	Ahmedabad	NaN	61,000	West
107	Pooja Das	Kolkata	14,000	NaN	East
108	Riya Kapoor	Jaipur	16,000	69,000	North

3. Mention how many records were lost
   Only 1 Record was lost which is of customer_ID: 105 which is:

Customer_ID	Name	City	Monthly_Sales	Income	Region
105	Amit Verma	Pune	18,000	58,000	NaN

Q9. Imputation

Handle missing values in Monthly_Sales using:

Forward Fill

Tasks:

1. Apply forward fill

Customer_ID	Name	City	Monthly_Sales	Income	Region
101	Rahul Mehta	Mumbai	12,000	65,000	West
102	Anjali Rao	Bengaluru	12,000		South
103	Suresh Iyer	Chennai	15,000	72,000	South
104	Neha Singh	Delhi			North
106	Karan Shah	Ahmedabad		61,000	West
107	Pooja Das	Kolkata	14,000		East
108	Riya Kapoor	Jaipur	16,000	69,000	North

2. Show before vs after values

Before Fill

Customer_ID	Name	City	Monthly_Sales	Income	Region
101	Rahul Mehta	Mumbai	12,000	65,000	West
102	Anjali Rao	Bengaluru	12,000		South
103	Suresh Iyer	Chennai	15,000	72,000	South
104	Neha Singh	Delhi			North
106	Karan Shah	Ahmedabad		61,000	West
107	Pooja Das	Kolkata	14,000		East
108	Riya Kapoor	Jaipur	16,000	69,000	North

After Fill

Customer_ID	Name	City	Monthly_Sales	Income	Region
101	Rahul Mehta	Mumbai	12,000	65,000	West
102	Anjali Rao	Bengaluru	12,000		South
103	Suresh Iyer	Chennai	15,000	72,000	South
104	Neha Singh	Delhi	15,000		North
106	Karan Shah	Ahmedabad	15,000	61,000	West
107	Pooja Das	Kolkata	14,000		East
108	Riya Kapoor	Jaipur	16,000	69,000	North

3. Explain why forward fill is suitable here

   The data is **customer records entered sequentially** so the previous value is the most logical estimate for a missing one

   Missing values are **isolated single gaps** not large chunks making forward fill accurate and reliable

   No rows are deleted and no complex calculation is needed it is the simplest and most practical fix for this dataset

Q10. Flagging Missing Data

Create a flag column for missing Income.

Tasks:

1. Create Income_Missing_Flag (0 = present, 1 = missing)

Customer_ID	Name	City	Monthly_Sales	Income	Region	Income missing flag
101	Rahul Mehta	Mumbai	12,000	65,000	West	0
102	Anjali Rao	Bengaluru	12,000		South	1
103	Suresh Iyer	Chennai	15,000	72,000	South	0
104	Neha Singh	Delhi	15,000		North	1
106	Karan Shah	Ahmedabad	15,000	61,000	West	0
107	Pooja Das	Kolkata	14,000		East	1
108	Riya Kapoor	Jaipur	16,000	69,000	North	0

2. Show updated dataset

Customer_ID	Name	City	Monthly_Sales	Income	Region	Income missing flag
101	Rahul Mehta	Mumbai	12,000	65,000	West	0
102	Anjali Rao	Bengaluru	12,000		South	1
103	Suresh Iyer	Chennai	15,000	72,000	South	0
104	Neha Singh	Delhi	15,000		North	1
106	Karan Shah	Ahmedabad	15,000	61,000	West	0
107	Pooja Das	Kolkata	14,000		East	1
108	Riya Kapoor	Jaipur	16,000	69,000	North	0

3. Count how many customers have missing income

Count of columns having missing income	3